

PRACTICAL FILE

Information Security

Bachelor of Science (Honours) in Computer Science



“Only Knowledge can provide salvation”

SUBMITTED TO: Mr. Sumit Baberwal

SUBMITTED BY:
Meghanshi
16050

1. Demonstrate the use of Network tools: ping, ipconfig, ifconfig, tracert, arp, netstat, whois.

```
C:\Users\HP445>ipconfig /all
```

```
Windows IP Configuration
```

```
Host Name . . . . . : DESKTOP-PRPF23F
Primary Dns Suffix . . . . . : kmv.in
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
DNS Suffix Search List. . . . . : kmv.in
```

```
Ethernet adapter Ethernet 2:
```

```
Connection-specific DNS Suffix . : kmv.in
Description . . . . . : Realtek PCIe GbE Family Controller #2
Physical Address. . . . . : 10-62-E5-0B-00-B1
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::931c:a66e:2266:8389%22(Preferred)
IPv4 Address. . . . . : 192.168.4.10(Preferred)
Subnet Mask . . . . . : 255.255.248.0
Lease Obtained. . . . . : 12 February 2026 13:27:24
Lease Expires . . . . . : 13 February 2026 13:27:21
Default Gateway . . . . . : 192.168.0.1
DHCP Server . . . . . : 192.168.0.1
DHCPv6 IAID . . . . . : 403727077
DHCPv6 Client DUID. . . . . : 00-01-00-01-2F-56-F6-1F-A4-5D-36-CA-BD-BA
DNS Servers . . . . . : 192.168.2.184
                        103.8.45.5
                        8.8.8.8
NetBIOS over Tcpip. . . . . : Enabled
```

```
C:\Users\HP445>ping 8.8.8.8
```

```
Pinging 8.8.8.8 with 32 bytes of data:
```

```
Reply from 8.8.8.8: bytes=32 time=18ms TTL=120
```

```
Reply from 8.8.8.8: bytes=32 time=13ms TTL=120
```

```
Reply from 8.8.8.8: bytes=32 time=15ms TTL=120
```

```
Request timed out.
```

```
Ping statistics for 8.8.8.8:
```

```
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
```

```
Approximate round trip times in milli-seconds:
```

```
    Minimum = 13ms, Maximum = 18ms, Average = 15ms
```

```

Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:

  1    <1 ms    <1 ms    <1 ms    192.168.1.1
  2     1 ms     1 ms     1 ms    del-pp-bngs-05 [205.254.165.8]
  3     4 ms     2 ms     2 ms    205.254.165.1
  4    14 ms     9 ms     9 ms    205.254.165.41
  5     4 ms     4 ms     4 ms    72.14.208.36
  6     3 ms     2 ms     2 ms    142.251.66.169
  7     3 ms     3 ms     2 ms    142.251.49.115
  8     3 ms     3 ms     2 ms    dns.google [8.8.8.8]

Trace complete.

```

```

$ whois --help
Usage: whois [OPTION]... OBJECT...

-h HOST, --host HOST    connect to server HOST
-p PORT, --port PORT    connect to PORT
-I                      query whois.iana.org and follow its referral
-H                      hide legal disclaimers
    --verbose           explain what is being done
    --no-recursion      disable recursion from registry to registrar servers
    --help              display this help and exit
    --version           output version information and exit

These flags are supported by whois.ripe.net and some RIPE-like servers:
-l                  find the one level less specific match
-L                  find all levels less specific matches
-m                  find all one level more specific matches
-M                  find all levels of more specific matches
-c                  find the smallest match containing a mnt-irt attribute
-x                  exact match
-b                  return brief IP address ranges with abuse contact
-B                  turn off object filtering (show email addresses)
-G                  turn off grouping of associated objects
-d                  return DNS reverse delegation objects too
-i ATTR[,ATTR]...   do an inverse look-up for specified ATTRIBUTES
-T TYPE[,TYPE]...   only look for objects of TYPE
-K                  only primary keys are returned
-r                  turn off recursive look-ups for contact information
-R                  force to show local copy of the domain object even
                    if it contains referral
-a                  also search all the mirrored databases
-s SOURCE[,SOURCE]... search the database mirrored from SOURCE
-g SOURCE:FIRST-LAST find updates from SOURCE from serial FIRST to LAST
-t TYPE             request template for object of TYPE
-v TYPE             request verbose template for object of TYPE
-q [version|sources|types] query specified server info

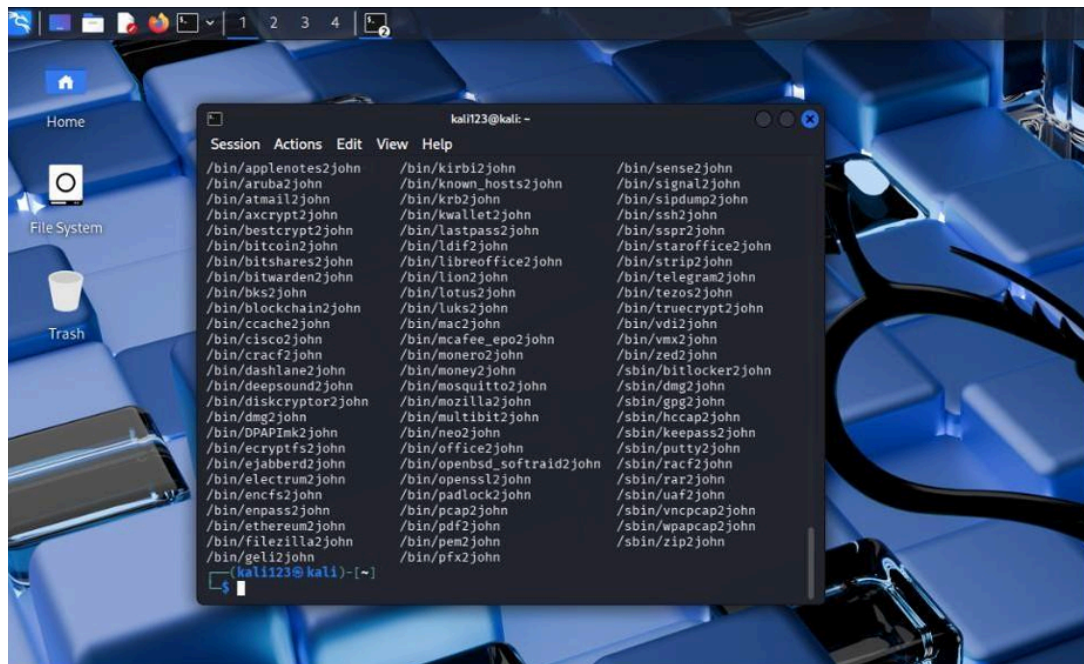
```

```
C:\Users\my pc>netstat -ano

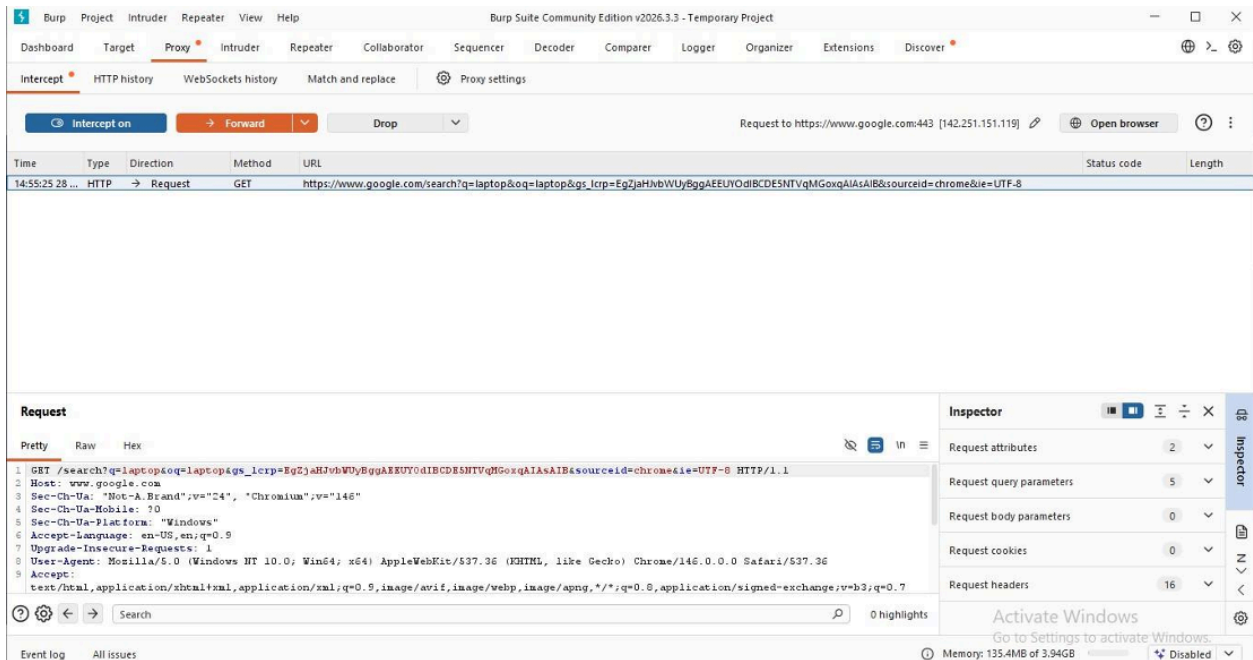
Active Connections

Proto Local Address          Foreign Address         State       PID
TCP   0.0.0.0:135             0.0.0.0:0               LISTENING   560
TCP   0.0.0.0:445             0.0.0.0:0               LISTENING   4
TCP   0.0.0.0:902             0.0.0.0:0               LISTENING   3456
TCP   0.0.0.0:912             0.0.0.0:0               LISTENING   3456
TCP   0.0.0.0:5040            0.0.0.0:0               LISTENING   6744
TCP   0.0.0.0:5357            0.0.0.0:0               LISTENING   4
TCP   0.0.0.0:5432            0.0.0.0:0               LISTENING   4116
TCP   0.0.0.0:7680            0.0.0.0:0               LISTENING   11196
TCP   0.0.0.0:24563           0.0.0.0:0               LISTENING   11348
TCP   0.0.0.0:49664           0.0.0.0:0               LISTENING   836
TCP   0.0.0.0:49665           0.0.0.0:0               LISTENING   664
TCP   0.0.0.0:49666           0.0.0.0:0               LISTENING   1296
TCP   0.0.0.0:49667           0.0.0.0:0               LISTENING   1740
TCP   0.0.0.0:49668           0.0.0.0:0               LISTENING   3028
TCP   0.0.0.0:49669           0.0.0.0:0               LISTENING   808
TCP   0.0.0.0:52161           0.0.0.0:0               LISTENING   10676
TCP   127.0.0.1:80            0.0.0.0:0               LISTENING   10924
TCP   127.0.0.1:1883          0.0.0.0:0               LISTENING   10924
TCP   127.0.0.1:9392          0.0.0.0:0               LISTENING   10924
TCP   192.168.0.154:139       0.0.0.0:0               LISTENING   4
```

2. Use of Password cracking tools: John the Ripper, Ophcrack. Verify the strength of passwords using these tools.



4. Use Burp proxy to capture and modify the message.



5. Implement caesar cipher substitution operation.

CODE-

```
def caesar_encrypt(text, shift):  
    result = ""
```

```
    for ch in text:  
        if ch.isalpha():  
            start = ord('A') if ch.isupper() else ord('a')  
            result += chr((ord(ch) - start + shift) % 26 + start)  
        else:  
            result += ch
```

```
    return result
```

```
def caesar_decrypt(text, shift):  
    return caesar_encrypt(text, -shift)
```

```
msg = input("Enter message: ")  
shift = int(input("Enter shift: "))
```

```
enc = caesar_encrypt(msg, shift)  
print("Encrypted:", enc)
```

```
dec = caesar_decrypt(enc, shift)
print("Decrypted:", dec)
```

OUTPUT -

```
Enter message: meoww meoowww
Enter shift: 2
Encrypted: ogqyy ogqqyyy
Decrypted: meoww meoowww
```

6. Implement monoalphabetic and polyalphabetic cipher substitution operation.

CODE-

```
#monoalphabetic
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
key = "QWERTYUIOPASDFGHJKLZXCVBNM"
```

```
def mono_encrypt(text):
    result = ""

    for ch in text.upper():
        if ch in alphabet:
            result += key[alphabet.index(ch)]
        else:
            result += ch

    return result
```

```
def mono_decrypt(text):
    result = ""

    for ch in text.upper():
        if ch in key:
            result += alphabet[key.index(ch)]
        else:
            result += ch

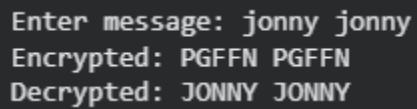
    return result
```

```
msg = input("Enter message: ")
```

```
enc = mono_encrypt(msg)
print("Encrypted:", enc)
```

```
dec = mono_decrypt(enc)
print("Decrypted:", dec)
```

OUTPUT-



```
Enter message: jonny jonny
Encrypted: PGFFN PGFFN
Decrypted: JONNY JONNY
```

```
# polyalphabetic
def vigenere_encrypt(text, key):
    result = ""
    key = key.upper()
    k = 0

    for ch in text.upper():
        if ch.isalpha():
            shift = ord(key[k % len(key)]) - ord('A')
            result += chr((ord(ch) - ord('A') + shift) % 26 + ord('A'))
            k += 1
        else:
            result += ch

    return result

def vigenere_decrypt(text, key):
    result = ""
    key = key.upper()
    k = 0

    for ch in text.upper():
        if ch.isalpha():
            shift = ord(key[k % len(key)]) - ord('A')
            result += chr((ord(ch) - ord('A') - shift) % 26 + ord('A'))
            k += 1
        else:
            result += ch

    return result
```

```

msg = input("Enter message: ")
key = input("Enter key: ")

enc = vigenere_encrypt(msg, key)
print("Encrypted:", enc)

dec = vigenere_decrypt(enc, key)
print("Decrypted:", dec)

```

OUTPUT -

```

Enter message: jonny jonny
Enter key: meow
Encrypted: VSBJK NCJZC
Decrypted: JONNY JONNY

```

7. Implement playfair cipher substitution operation.

CODE-

```

def generate_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []
    used = set()

    for ch in key:
        if ch not in used and ch.isalpha():
            matrix.append(ch)
            used.add(ch)

    for ch in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
        if ch not in used:
            matrix.append(ch)

    return [matrix[i:i+5] for i in range(0, 25, 5)]

def find_pos(matrix, ch):
    for i in range(5):
        for j in range(5):
            if matrix[i][j] == ch:
                return i, j

```

```

def prepare(text):
    text = text.upper().replace("J", "I")
    text = ".join(filter(str.isalpha, text))

    if len(text) % 2 != 0:
        text += 'X'

    return text

def playfair_encrypt(text, key):
    matrix = generate_matrix(key)
    text = prepare(text)

    result = ""

    for i in range(0, len(text), 2):
        a, b = text[i], text[i+1]

        r1, c1 = find_pos(matrix, a)
        r2, c2 = find_pos(matrix, b)

        if r1 == r2:
            result += matrix[r1][(c1+1)%5]
            result += matrix[r2][(c2+1)%5]

        elif c1 == c2:
            result += matrix[(r1+1)%5][c1]
            result += matrix[(r2+1)%5][c2]

        else:
            result += matrix[r1][c2]
            result += matrix[r2][c1]

    return result

def playfair_decrypt(text, key):
    matrix = generate_matrix(key)
    result = ""

    for i in range(0, len(text), 2):
        a, b = text[i], text[i+1]

        r1, c1 = find_pos(matrix, a)
        r2, c2 = find_pos(matrix, b)

```

```

if r1 == r2:
    result += matrix[r1][(c1-1)%5]
    result += matrix[r2][(c2-1)%5]

elif c1 == c2:
    result += matrix[(r1-1)%5][c1]
    result += matrix[(r2-1)%5][c2]

else:
    result += matrix[r1][c2]
    result += matrix[r2][c1]

return result

```

```

msg = input("Enter message: ")
key = input("Enter key: ")

enc = playfair_encrypt(msg, key)
print("Encrypted:", enc)

dec = playfair_decrypt(enc, key)
print("Decrypted:", dec)

```

OUTPUT-

```

Enter message: appletree
Enter key: orange
Encrypted: OTHDPOBCV
Decrypted: APPLETREEX

```

8. Implement hill cipher substitution operation.

CODE-

```

import numpy as np
key = np.array([[3, 3],
                [2, 5]])

inverse_key = np.array([[15, 17],
                        [20, 9]])

def hill_encrypt(text):
    text = text.upper().replace(" ", "")

```

```

if len(text) % 2 != 0:
    text += 'X'

result = ""

for i in range(0, len(text), 2):
    pair = [
        ord(text[i]) - 65,
        ord(text[i+1]) - 65
    ]

    enc = np.dot(key, pair) % 26

    result += chr(int(enc[0]) + 65)
    result += chr(int(enc[1]) + 65)

return result

def hill_decrypt(text):
    result = ""

    for i in range(0, len(text), 2):
        pair = [
            ord(text[i]) - 65,
            ord(text[i+1]) - 65
        ]

        dec = np.dot(inverse_key, pair) % 26

        result += chr(int(dec[0]) + 65)
        result += chr(int(dec[1]) + 65)

    return result

msg = input("Enter message: ")
enc = hill_encrypt(msg)
print("Encrypted:", enc)

dec = hill_decrypt(enc)
print("Decrypted:", dec)

```

OUTPUT-

```
Enter message: newton ate that apple
Encrypted: ZUTJDPFRRZVOFMMBTQ
Decrypted: NEWTONATETHATAPPLE
```

9. Implement rail fence cipher transposition operation.

CODE-

```
def encrypt(text, key):
    rail = [" " for _ in range(key)]
    row, step = 0, 1

    for ch in text:
        rail[row] += ch
        if row == 0:
            step = 1
        elif row == key - 1:
            step = -1
        row += step

    return "".join(rail)

def decrypt(cipher, key):
    n = len(cipher)
    rail = [" " for _ in range(n)]

    row, step = 0, 1
    for i in range(n):
        rail[row][i] = '*'
        if row == 0:
            step = 1
        elif row == key - 1:
            step = -1
        row += step

    index = 0
    for i in range(key):
        for j in range(n):
            if rail[i][j] == '*':
                rail[i][j] = cipher[index]
                index += 1

    # read
    result = ""
    row, step = 0, 1
```

```

for i in range(n):
    result += rail[row][i]
    if row == 0:
        step = 1
    elif row == key - 1:
        step = -1
    row += step

return result

txt=input(str("enter the text: "))
k=int(input("enter the key: "))
print("original text = ", txt)
p=encrypt(txt,k)
print("encrypted text = ", p)
print("decrypted text = ", decrypt(p,k))

```

Output:-

```

enter the text: kat kite kut
enter the key: 3
original text =  kat kite kut
encrypted text =  kk a iektttu
decrypted text =  kat kite kut

```

10. Implement row transposition cipher transposition operation.

CODE-

```

def row_encrypt(text, key):
    cols = len(key)
    rows = -(-len(text) // cols)
    text += 'X' * (rows * cols - len(text))

    matrix = [list(text[i:i+cols]) for i in range(0, len(text), cols)]
    order = sorted(range(cols), key=lambda x: key[x])

    return "".join(matrix[r][c] for c in order for r in range(rows))

def row_decrypt(cipher, key):
    cols = len(key)
    rows = len(cipher) // cols

    order = sorted(range(cols), key=lambda x: key[x])
    matrix = ["" * cols for _ in range(rows)]

    index = 0

```

```

for c in order:
    for r in range(rows):
        matrix[r][c] = cipher[index]
        index += 1

    return ".join(matrix[r][c] for r in range(rows) for c in range(cols))
text=input(str("enter the text: "))
keY=int(input("enter the key: "))
print("original text = ", text)
p=row_encrypt(text, key)
print("encrypted text = ", p)
print("decrypted text = ", row_decrypt(cipher, key))

```

OUTPUT-

```

enter the text: hello
enter the key: 1234
original text =  hello
encrypted text =  eXlXholX
decrypted text =  HELLOWORLDXX

```

11. Implement product cipher transposition operation.

CODE-

Product Cipher using Double Transposition

```

def transpose_encrypt(text, key):
    result = ""

```

```

    for col in range(key):
        pointer = col

```

```

        while pointer < len(text):
            result += text[pointer]
            pointer += key

```

```

    return result

```

```

def transpose_decrypt(cipher, key):
    n = len(cipher)
    rows = (n + key - 1) // key

    grid = [""] * n
    index = 0

```

```
for col in range(key):
    pointer = col

    while pointer < n:
        grid[pointer] = cipher[index]
        index += 1
        pointer += key

return ".join(grid)
```

```
# Input
message = input("Enter message: ")

key1 = 4
key2 = 3

# Encryption
step1 = transpose_encrypt(message, key1)
cipher = transpose_encrypt(step1, key2)

print("Encrypted Text:", cipher)

# Decryption
step2 = transpose_decrypt(cipher, key2)
plain = transpose_decrypt(step2, key1)

print("Decrypted Text:", plain)
```

OUTPUT-

```
Enter message: meoww meoww meowww
Encrypted Text: mm wwwwwwoo oeemww
Decrypted Text: meoww meoww meowww
```